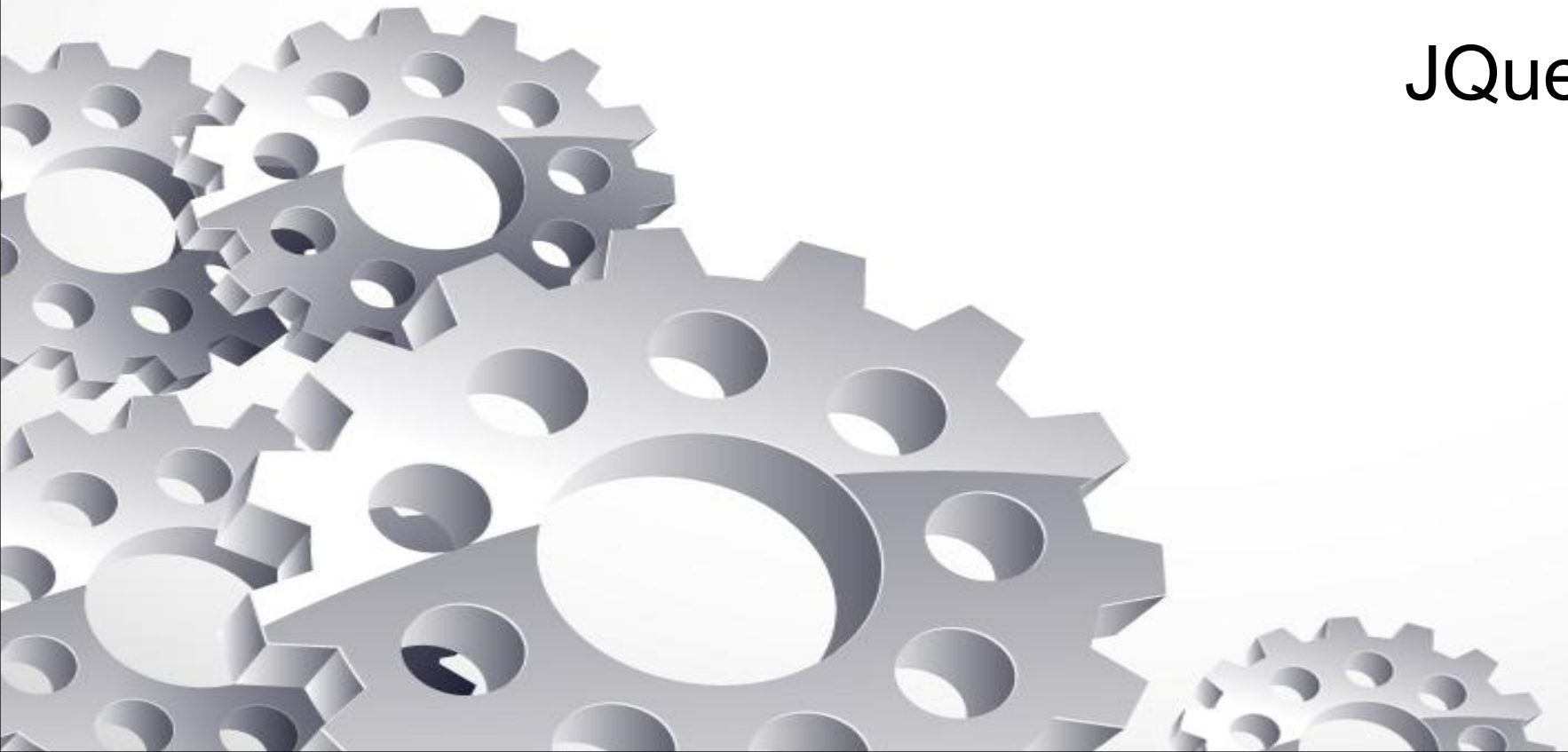
# Web Technology SCSJ 3483

## JQuery Ajax Method



# jQuery Ajax?



- jQuery Ajax methods provide an easy-to-use API for making asynchronous requests, handling responses, and updating the HTML content of a web page dynamically.
- Can send and receive data in various formats such as JSON, XML, HTML, or plain text
- Can handle different types of HTTP requests such as GET, POST, PUT, DELETE, and others
- jQuery Ajax is a powerful tool for building modern web applications that can interact with web servers in a fast, flexible, and efficient manner

# $.ajax configuration parameters



some of the most commonly used parameters of $.ajax({}):

- **url (string, required):** The URL of the web server endpoint that you want to send the request to.
- **type (string, optional):** The HTTP method to use for the request, such as "GET", "POST", "PUT", "DELETE" etc (default value is "GET").
- **data (object or string, optional):** The data to send to the server with the request. This can be a plain object, a URL-encoded string, or a JSON string.
- **dataType (string, optional):** The expected data type of the response from the server, such as "json", "xml", "html", "text" etc.
- **success (function, optional):** The callback function to execute when the request is successful. The function is passed the response data as its first argument.
- **error (function, optional):** The callback function to execute when the request fails. The function is passed the XHR object, the error message, and the error type as its arguments.

# Example : jQuery Ajax to make GET request

```
<!DOCTYPE html><html>
<head>
  <title>jQuery Ajax Example</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <div id="result"></div>
  <script>
    $(document).ready(function() {
        $.ajax({
          url: 'https://jsonplaceholder.typicode.com/todos/1',
          type: 'GET',
          success: function(data) {
            $('#result').html('<p>' + JSON.stringify(data) + '</p>');
          },
          error: function() {
            $('#result').html('<p>Error fetching data</p>');
          }
        });
    });
  </script></body></html>
```

# Example : jQuery Ajax to make POST request

```
<body>
  <form>
    <input type="text" id="username" placeholder="Enter your username">
    <input type="text" id="password" placeholder="Enter your password">
    <button type="button" id="login-btn">Login</button>
  </form>
  <div id="result"></div>
  <script>
    $(document).ready(function() {
      $('#login-btn').click(function() {
        var username = $('#username').val();
        var password = $('#password').val();
        $.ajax({
          url: '/login',
          type: 'POST',
          data: {
            username: username,
            password: password
          },
          success: function(data) {
            $('#result').html('<p>' + data.message + '</p>');
          },
          error: function() {
            $('#result').html('<p>Error logging in</p>');
          }
        });
      });
    });
  </script>
</body>
```

# $.ajax() alternatives



- $.get(), $.post() and $.getJSON()
- These alternative methods provided by jQuery library that allow us to make an Ajax request to a web server.
- Differ in terms of syntax, default settings, and use cases

# $.ajax({})

- the most flexible Ajax method provided by jQuery.
- allow to specify all of the parameters for the Ajax request, including the URL, HTTP method, data to send, and expected data type of the response. This method can be used for all types of Ajax requests, including GET, POST, PUT, DELETE etc.
- It is typically used when you need fine-grained control over the Ajax request and response

```
$.ajax({
  url: "https://api.example.com/data",
  type: "GET",
  success: function(response) {
    console.log(response);
  },
  error: function(xhr, status, error) {
    console.log("Error: " + error);
  }
});
```

# $.get()



is a shorthand method provided by jQuery for making GET requests.

It takes two parameters: the URL of the web server endpoint to send the request to, and an optional callback function to execute when the request is successful.

By default, $.get() assumes that the response data will be in plain text format, but you can specify the expected data type using the dataType parameter.

```
$.get('https://api.example.com/data', function(data) {
  console.log(data);
}).fail(function(error) {
  console.error(error);
});
```

# $.post()

is a shorthand method provided by jQuery for making POST requests.

It takes three parameters: the URL of the web server endpoint to send the request to, the data to send with the request, and an optional callback function to execute when the request is successful.

By default, $.post() assumes that the response data will be in plain text format, but you can specify the expected data type using the dataType parameter.

```
$.post("https://api.example.com/data", { name: "John", age: 30 }, function(re
  console.log(response);
});
```

# $.getJSON()

- is a shorthand method provided by jQuery for making GET requests and expecting the response data to be in JSON format.
- It takes two parameters: the URL of the web server endpoint to send the request to, and an optional callback function to execute when the request is successful.

```
$.getJSON("https://api.example.com/data", function(response) {
  console.log(response);
});
```



end